

Docs Translator CLI

Разработка CLI-утилит с использованием ИИ-инструментов



ChatGPT vs Claude

Практическое задание

Цель

- Освоить разработку CLI-утилит с ИИ-инструментами
- Сравнить подходы двух нейросетей

Ключевые требования

- ≥ 7 команд/подкоманд
- Обязательный REST API
- 2 разные нейросети
- Тесты + покрытие $> 60\%$
- Сохранить промпты

5 этапов

- Выбор темы
- Подготовка ТЗ
- Разработка
- Отчёт
- GitHub

Проект: Docs Translator CLI

Консольная Python-утилита для перевода текстовой документации через LibreTranslate-compatible REST API. Локальное хранение истории переводов и пользовательского глоссария терминов.

12 команд · 2 реализации · 1 ТЗ

ChatGPT → version1
Claude → version2

REST API

LibreTranslate /translate /detect /languages /health

Глоссарий

SQLite-хранилище терминов с placeholder-механизмом

История

Запись переводов, экспорт JSON/CSV/Markdown

Конфигурация

CLI > env > JSON-файл > defaults

Markdown-защита

Code blocks, inline code и URL не переводятся

doctor

Диагностика окружения и доступности API

Технологический стек

Python 3.12

Основной язык

Typer

CLI-фреймворк

HTTPX

HTTP-клиент для REST API

pytest + respx

Тесты с мок-запросами

hatchling

Сборка Python-пакета

uv

Менеджер пакетов и запуска

Rich

Форматированный вывод

SQLite

История + глоссарий

ruff

Линтинг и форматирование

platformdirs

Системные пути

12 реализованных CLI-команд (требование: ≥ 7)

Команда	API / хранилище	Назначение
<code>translate-text</code>	POST /translate	Перевод строки
<code>translate-file</code>	POST /translate	Перевод .txt/.md/.rst файла
<code>detect-language</code>	POST /detect	Определение языка
<code>languages-list</code>	GET /languages	Список доступных языков
<code>glossary-add</code>	SQLite	Добавить термин в глоссарий
<code>glossary-list</code>	SQLite	Просмотр глоссария
<code>glossary-remove</code>	SQLite	Удалить термин
<code>history-list</code>	SQLite	История переводов
<code>export-history</code>	SQLite + FS	Экспорт JSON/CSV/Markdown
<code>config-set</code>	JSON файл	Сохранить настройку
<code>config-show</code>	JSON файл	Просмотр конфигурации
<code>doctor</code>	GET /health	Диагностика окружения и API

REST API интеграция

LibreTranslate API — Endpoints

POST **/translate**

Перевод текста/сегментов

POST **/detect**

Определение языка

GET **/languages**

Список языков

GET **/health**

Статус API (для doctor)



Конфигурация

```
base_url → http://localhost:5000
api_key, timeout, default_source
default_target, log_level
```



Локальный LibreTranslate

```
docker run -it --rm \
  -p 5000:5000 \
  libretranslate/libretranslate
```



Все тесты используют respx-моки — реальные HTTP-запросы не выполняются, что делает CI стабильным

Разработка в ChatGPT

1

Итерация 1: ТЗ → архив

25.05.2026 15:11 • ~8 мин

- ✓ 12 команд, SQLite, REST-клиент, README
- ✓ 22 теста (заявлено: 80% покрытие)
- ✗ `uv sync` не собирал проект
- ✗ Нехватало build-конфигурации и `typing-extensions`

2

Итерация 2: исправление

25.05.2026 20:53 • быстро

- ✓ Добавлен hatchling + явный `package src/docstrans`
- ✓ Исправлено закрытие SQLite connection

3

Ruff SIM105

Финальная проверка

- ✓ Исправлен `try-except-pass` → `contextlib.suppress`
- ✓ Все проверки пройдены

Разработка в Claude (Anthropic)

1

Итерация 1: ТЗ → реализация

25.05.2026 15:06 • ~10 мин

- ✓ Систематически: зависимости → код → CLI → тесты → README
- ✗ Закончился лимит инструментов — архив не упакован

2

Итерация 2: "Continue" → завершение

25.05.2026 15:27 • ~20 мин

- ✓ 45/45 тестов, покрытие 69%
- ✓ ruff: All checks passed! — без дополнительных правок
- ✓ Подробные help-описания, README на русском
- ✗ В архиве лишний пустой каталог {src (не влияет на запуск)

3

Подготовка к GitHub

Финальная очистка

- ✓ Удалён {src, всё работает корректно

Результаты проверки

22

ТЕСТОВ
ChatGPT

45

ТЕСТОВ
Claude

80%

покрытие
ChatGPT

69%

покрытие
Claude

12

команд
обе версии

Критерий	ChatGPT	Claude
uv sync	✓ после 2 итераций	✓ после Continue
pytest	22 passed • 80%	45 passed • 69%
ruff check	✓ после SIM105 fix	✓ сразу
ruff format	✓ 37 files formatted	✓ 37 files formatted
CLI help	12 команд	12 команд
REST API doctor	ok	ok
translate-text	✓	✓
Структура архива	cache-файлы убраны	{src убран

Сравнительный анализ

Критерий	ChatGPT	Claude
Понимание контекста	★★★★★	★★★★★
Первая работоспособность	★★★★☆☆	★★★★★☆☆
Количество тестов	★★★★☆☆	★★★★★
Покрытие тестами	★★★★★	★★★★★☆☆
Чистота lint (ruff)	★★★★☆☆	★★★★★
Скорость ответа	★★★★★	★★★★★☆☆
Документация	★★★★☆☆	★★★★★
Поддержка диалога	★★★★☆☆	★★★★★☆☆

 Итог: оба инструмента справились. ChatGPT — выше покрытие (80%). Claude — больше тестов (45) и чистый ruff с первого раза.

Ключевые выводы



Подробное ТЗ — главный фактор успеха. «Сделай CLI» → провал; список команд, API, структура, тесты → рабочий проект с 1–2 итерации.



Нельзя верить отчёту нейросети без локальной проверки. `uv sync`, `pytest`, `ruff` — обязательный pipeline после получения архива.



Передавать реальный лог ошибок, а не пересказ. ChatGPT точно исправил packaging именно по `run_log.md`.



Для REST API без ключа — локальный LibreTranslate через Docker. Стабильно и воспроизводимо для демонстрации и тестов.



Оба инструмента применимы для CLI-проектов по чёткому ТЗ. Ни один результат нельзя принимать без human-review архива.

Практическая работа выполнена

✓ CLI использует REST API (LibreTranslate-compatible)

✓ Обе версии работают без ошибок

✓ 12 команд (требование ≥ 7)

✓ PEP 8 / ruff: All checks passed

✓ Тесты: ChatGPT 22 / Claude 45 • Покрытие: 80% / 69%

✓ Промпты и диалоги сохранены

✓ Детальный Markdown-отчёт оформлен

✓ Структура репозитория по требованию

github.com/stormy23dev/pyprompting-docstrans